

Problem 3: Simulated Annealing

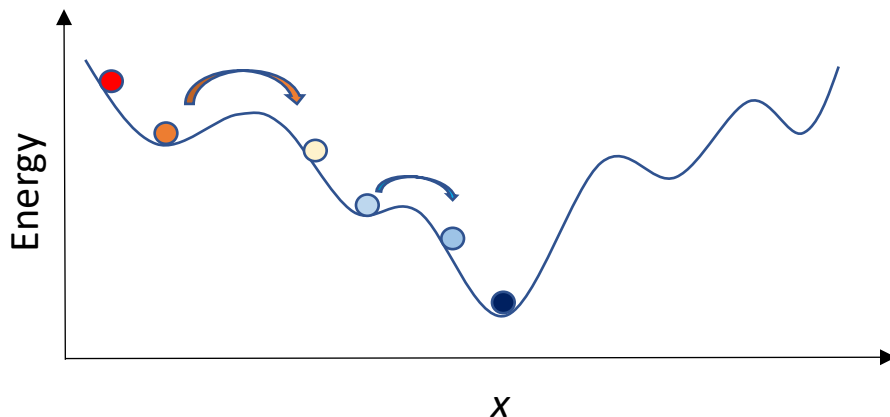
Global optimization

In a previous activity guide, you looked at finding the location of an extreme value of a function (let's consider that you are looking for a minimum). Consider the situation where the function has many local minima – how do we guarantee that we find the global minimum? Answer: there is no guaranteed way of solving such a problem in a finite amount of time; moreover, when the function being optimized is a function over many variables, with many minima, the likelihood gets even smaller. The `scipy` minimization method that we used was based on starting with some initial guess and then making moves based on how the function was changing. Most of these methods make moves that always lower the function, with the result being that one simply 'descends' to the closest local minimum and gets stuck there. To find the global minimum requires starting from many different initial points, hoping that you visit all the local minima, and asserting that the lowest one you visited is the global minimum.

In this project, you will explore a physics-inspired method called *simulated annealing* to find the global minimum, which borrows from our work using the Metropolis Monte Carlo algorithm. With the right move set, the system is ergodic (it can get to any state from any other) and so will eventually make a random (though potentially highly improbable) move to the global minimum. Thus the method should eventually find the global optimum, but this might take an impractically long amount of time.

Simulated Annealing

Imagine a physical system characterized by an energy function that governs the degrees of freedom of the objects making it up. When the system is at high temperature, it occupies high-energy configurations (e.g., liquid metal). Then imagine *annealing* it, i.e., gradually cooling it down: as the system cools down, it relaxes into lower-energy configurations (e.g., becoming more solid-like). If we cool it down sufficiently slowly, the system will relax into the ground-state energy configuration (e.g., a defect-free solid). This is distinguished from *quenching* where the temperature is lowered quickly and the system gets trapped in a high-energy configuration (that usually has lots of defects). A schematic of annealing is shown below.



In the figure, at high temperatures the system can make big thermal jumps over high energy barriers. As the system cools, the size of these jumps gets smaller, making it less likely to go over big barriers (or out of minima). This provides a prescription for how we could modify our Metropolis Monte Carlo method to carry out simulated annealing. The algorithm is:

1. Initialize the system into some initial configuration
2. Set the temperature at some 'hot' value
3. Do a Metropolis Monte Carlo sweep at the given temperature
4. Lower the temperature (using some temperature schedule)
5. Iterate: Repeat steps 3-4 until some final temperature is reached.
6. Quench: Do one final sweep at $T = 0$ (so only energy-lowering moves are accepted).

Thus you just have to wrap your Metropolis Monte Carlo algorithm in an external loop that lowers the temperature until some final temperature is reached. Of course, the art of simulated annealing is in picking the temperature-cooling schedule. This is problem dependent, and typically one must experiment (making adjustments to the schedule) to find a good minimum in a reasonable amount of simulation time.

Temperature Schedules

One choice of a temperature schedule is to update the temperature after each Monte-Carlo sweep according to:

$$T \leftarrow \alpha T$$

where $\alpha < 1$. Using an α near 1 (e.g., 0.92 or 0.9) cools the system quite slowly and is usually a good starting point. Again, this is something that should be experimented with for the given problem.

Applications: NP-hard problems, Binary Quadratic Models, and D-Wave

Many optimization problems in combinatorics and graph theory can be mapped onto a more general version of the Ising model known as a spin glass. For a spin glass, there is a set of interacting spins $\{s_i\}$ with energy

$$H = \sum_i h_i s_i + \sum_{i \neq j} J_{i,j} s_i s_j$$

Unlike in the Ising model, each spin can in principle interact with any other spin via the interaction $J_{i,j}$. The h 's represent a constant bias (such as a magnetic field). In mathematics and computer science, such a model is known as a *Binary Quadratic Model* (BQM) since the spins take on values of -1 or 1 (or potentially 0 and 1, depending on the problem). A given optimization problem corresponds to a specific set of h 's and J 's. The $\{s_i\}$ that minimizes the energy will then be a minimum (hopefully global) for the given problem.

Adiabatic quantum optimization and D-Wave

Why would one want to convert a problem into a BQM model? Such a model can be optimized using *adiabatic quantum optimization* (AQO). The idea behind AQO is to prepare a system in a ground state of a simple energy function H_0 and then gradually push the system towards the desired energy function H . If done slowly and at zero temperature, the system will stay in the ground state all the way along the path and thus yield the global optimal solution. The quantum-mechanical advantage is that the system can tunnel through energy barriers, which is not possible classically. D-Wave computers (started here in Burnaby) are built to do AQO. As of writing, the speed-up over solving the problem classically is not firmly established, as moving the system sufficiently slowly and at $T=0$ is technically not possible. So one can't guarantee to get the true ground state nor necessarily be faster than a classical calculation. But it's still a fascinating application. So if you can convert your optimization problem into a BQM, it can be solved on a D-Wave computer.

Application 1: Number Partitioning

Here's an NP-hard problem that can be recast as a spin-glass energy function to be minimized. Suppose you have N positive numbers (think of assets each with a certain \$ value) $[n_1, n_2, \dots, n_N]$ and you need to divide them up between 2 people such that the individual sums are as equal as possible. One could imagine labelling the assets belonging to one person (+1) and the other (-1). Then a good partitioning would minimize the energy (see reference [1])

$$H = \left(\sum_i n_i s_i \right)^2$$

- Analysis: Given how we have defined our s 's, why do we want this cost/energy function to be squared for the purpose of minimization?
- Analysis: Show that the above energy can be written as

$$H = \sum_i \sum_j J_{i,j} s_i s_j$$

where the interactions are $J_{i,j} = n_i n_j$.

- Code: Adapt your Monte Carlo code for the Ising Model (see Activity Guide 8) to do simulated annealing and minimize the above function. Your simulated-annealing code should take as input a starting temperature T_0 , a final temperature T_f , and the scale parameter α . For this problem, it will also take as input a given set of values for the n 's, which will allow you to calculate the interaction matrix J defined above. Your code should initialize the N binary spins to be a random configuration of ± 1 's. In each iteration of your Monte-Carlo sweep, the code should randomly flip one of the spins and apply the Metropolis-Hastings criterion to the change in energy. It should output the lowest-energy configuration as well as the lowest energy found.
- Application: Use $N=15$, with the n 's drawn randomly from integer values ranging from 1 to 100. What should your starting temperature be? And your final temperature? **TIP**: The energy scale of H goes as $J_{i,j} = n_i n_j$, and so jumps are only possible if $T > J$. What is your largest $n_i n_j$? Your starting temperature should be larger than that. You could try a final temperature of $1e-4 * (\max(n_i n_j))$.

- e. Run your anneal several times for the same set of n 's, and experiment with your scale parameter α . Can you get $H = 0$? if not, what's the smallest H you get? Are there different s 's (i.e., partitions) that give the same minimum value (i.e., are there several ways to distribute the assets) ?

Application 2: Traveling-Salesperson Problem (TSP):

Another optimization problem that can be recast as a BQM is the traveling-salesperson problem: starting from your home city, you need to visit each of $N-1$ other cities only once and return home, with the shortest total distance travelled. In the figure below, on the left is a possible path that is not the shortest-distance path, whereas on the right is the optimal path.



We'll solve this problem using a classical simulated-annealing approach, then lastly, you'll do the math to see how to convert it to a BQM that a D-Wave computer could solve.

To set up the problem, let's label the cities from 0 (your home city) to $N-1$, with positions $r_i = (X_i, Y_i)$ for each of the N cities. From these, the distance between two cities is $W_{u,v} = |r_u - r_v| = \sqrt{(X_u - X_v)^2 + (Y_u - Y_v)^2}$. A particular path is a vector $c = [c_0, c_1, \dots, c_N]$, where c_i is the city number at point i on the path. Each city's number can only occur once in a path, except for the beginning and end which are always the start city, that is $c_0 = 0$ and $c_N = 0$. For example, for 4 cities, $c = [0, 3, 1, 2, 0]$ would be a valid path. The total path distance (the 'energy' for purposes of minimization) is

$$H = \sum_{i=0}^{N-1} W_{c_i, c_{i+1}}$$

The goal is to find the path c that minimizes this total distance (energy).

- a. Modify your simulated-annealing codes to minimize the above distance function. You are given a set of N positions of cities. Your path is an array that contains the $N-1$ 'other' city numbers exactly once and begins and ends with the start city (take that to be 0). Your random move consists of picking 2 different elements of c and swapping their values (i.e., you swap two cities in the path). You then calculate the total distance of this new path and apply the Metropolis-Hastings criterion.
- b. Download the file 'cities_small.txt' from Canvas. It contains 2D positions for 10 cities. Run your simulated anneal and output the final path. Did you find the best path? **TIP:** Again, what should be a good starting temperature? It should be greater than the largest W .
- c. Now download the file 'cities_large.txt' from Canvas. This contains 30 cities. What's the best solution that you can find? Can you find the minimum path?

At face value, the above energy function doesn't look much like a BQM. But one can think about this as a BQM and hence the problem could be solved on a D-wave quantum-annealer computer

Converting the TSP into a BQM

The trick with converting the TSP into a BQM is coming up with a binary representation for the path (see [1]). This can be done using *one-hot encoding*: for a variable that can take on N values, you can turn this into a binary representation using now a 1D array of length N , with values = 0 except at the i th position if it has the i th value. So for example, for a variable that labels N cities, $c = 3$ can be represented with the vector $[0,0,1,0,0, \dots, 0]$. So the one-hot encoding of a particular path through the cities is now an $N \times (N+1)$ matrix. For example, the path $x = [0, 2, 1, 0]$ through $N = 3$ cities can be one-hot encoded by

$$x = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

So we've turned our problem of size N into a binary-variable problem of size $\sim N^2$. Our new binary variable can be summarized as $s_{v,i}$ where v is the city encoding and i is the position in the path.

The distance/energy can now be written in terms of this new variable as

$$H_B = \sum_{j=0}^{N-1} \sum_u \sum_v W_{u,v} s_{u,j} s_{v,j+1}$$

But this isn't all. We have constraints on what are acceptable paths, namely each city can only appear once and in the one-hot encoding there's only a single 1 to represent a given city. These can be added as additional energy costs,

$$H_A = \sum_v \left(1 - \sum_j s_{v,j} \right)^2 + \sum_j \left(1 - \sum_v s_{v,j} \right)^2$$

The first term adds a penalty if the one-hot encoding of a particular city is violated, and the second term ensures that each city is only visited once. The total energy to be minimized is

$$H = AH_A + BH_B$$

With the constraints satisfied when $0 < B \max(W_{u,v}) < A$, since then the maximum of the second term is always smaller than the first term.

1. Modify your simulated-annealing code to minimize the above function of the binary variables $s_{v,j}$. Apply your code to the above city files and check if the minimum one-hot encoded path is the same as you found previously.

This energy function is a BQM and can be simulated on a D-wave computer.

References:

[1] Andrew Lucas, "Ising formulations of many NP problems", <https://doi.org/10.3389/fphy.2014.00005>.